

"Code Se Developer Ki Soch Samajhna"

Document:	Code Archaeology — Feature PRD v1.0
Feature:	Core Mode 3 — Reverse Engineering Developer Thinking
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

Code Archaeology IdeaDNA ka **sabse unique feature** hai. Koi bhi code snippet, GitHub repo, ya live website daalne pe AI **developer ki thinking process reverse-engineer** karta hai. Kya problem thi? Kaise socha? Kya trade-offs kiye? Kya future plan tha? — sab pata chalta hai.

Unique Value: No other platform does this. ChatGPT code explain karta hai, par 'developer ne ye kyun socha?' nahi batata. Hum psychology + architecture + history — teeno level pe analyze karte hain.

2. User Flow & Interaction Design

- **Step 1 — Mode Selection:** User dashboard pe ■ 'Analyze Code' select karta hai
- **Step 2 — Input:** 3 options: (a) Paste code directly, (b) GitHub repo URL, (c) Live website URL
- **Step 3 — Code Parsing:** System language, framework, architecture detect karta hai
- **Step 4 — Pattern Detection:** Design patterns, API patterns, security patterns identify hote hain
- **Step 5 — History Reconstruction:** Original problem → v1 → v2 → v3 evolution timeline banata hai
- **Step 6 — Psychology Analysis:** 'Developer ne ye kyun socha?' — trade-offs, priorities, constraints, regrets
- **Step 7 — Scoring:** Innovation score, complexity, technical debt score
- **Step 8 — Future Evolution:** Suggested next versions based on current architecture
- **Step 9 — Action:** Evolve this idea, Generate blueprint, or Save to library

3. 6-Agent Pipeline Architecture

Agent 1: CodeParserAgent

Temperature: 0.3. Input: Raw code / GitHub URL / Website URL. Output: Language detection, file structure, components, dependencies, architecture pattern. Uses Tree-sitter for AST parsing + LLM for high-level analysis.

Agent 2: PatternDetectorAgent

Temperature: 0.3. Input: CodeParser output. Output: Design patterns (Factory, Singleton, Observer), API patterns (REST, GraphQL, gRPC), Security patterns (JWT, OAuth, RBAC), Performance patterns (Caching, CDN, Lazy Loading). Confidence score har pattern ke saath.

Agent 3: IdeaHistorianAgent

Temperature: 0.5. Input: Code structure + patterns. Output: Reconstructed evolution timeline — v1 (MVP), v2 (features added), v3 (current), v4 (planned but missing). Technical debt identification.

Agent 4: DevPsychologistAgent

Temperature: 0.4. Input: History + patterns. Output: Thinking process analysis — 'Why this tech stack?', 'What trade-offs?', 'What constraints?', 'What regrets?', 'What would they do differently now?' Ye sabse unique agent hai.

Agent 5: ScorerAgent

Temperature: 0.2. Input: Complete analysis. Output: Innovation score, technical debt score, maintainability score, scalability score.

Agent 6: EvolverAgent

Temperature: 0.6. Input: Complete analysis + scores. Output: 3-4 suggested future versions based on current architecture's strengths and weaknesses.

4. Input Handling — 3 Types

Type 1: Raw Code Snippet

User directly HTML/CSS/JS/Python paste karta hai. CodeParserAgent Tree-sitter se AST parse karta hai. File structure nahi hoti, isliye LLM se 'likely structure' infer karta hai.

Type 2: GitHub Repository URL

System GitHub API se repo fetch karta hai: README, package.json, main files, directory structure. Git history se commit patterns analyze karte hain (v1, v2, v3 reconstruction ke liye).

Type 3: Live Website URL

System Puppeteer/Playwright se website scrape karta hai: DOM structure, CSS classes, JS files, network requests. Static analysis se framework detect karta hai (React ke liye `__reactRootContainer`, Vue ke liye `__VUE__`, etc.).

GitHub API Integration

```
// Fetch repo structure from GitHub async function fetchGitHubRepo(repoUrl) { const
[owner, repo] = parseRepoUrl(repoUrl); // Get repo info const repoInfo = await
octokit.repos.get({ owner, repo }); // Get file tree const tree = await
octokit.git.getTree({ owner, repo, tree_sha: repoInfo.data.default_branch,
recursive: true }); // Get README const readme = await octokit.repos.getReadme({
owner, repo }); // Get recent commits for evolution timeline const commits = await
octokit.repos.listCommits({ owner, repo, per_page: 30 }); // Get package.json /
requirements.txt / etc. const packageJson = await fetchFile(owner, repo,
'package.json'); return { repoInfo, tree, readme, commits, packageJson }; }
```

5. Complete Output Structure

```
{ "codeAnalysis": { "languages": ["React", "Node.js", "TypeScript"], "framework": "Next.js 14", "architecture": "Monolithic with API Routes", "components": ["Auth", "Dashboard", "Payment", "Admin Panel"], "dependencies": ["stripe", "next-auth", "prisma", "tailwindcss"], "complexity": "MEDIUM", "linesOfCode": 15000 }, "patterns": { "designPatterns": [ { "pattern": "Repository Pattern", "confidence": 0.85, "location": "/lib/prisma/" }, { "pattern": "Factory Pattern", "confidence": 0.72, "location": "/components/ui/" } ], "apiPattern": { "type": "REST", "confidence": 0.95 }, "securityPatterns": [ { "pattern": "JWT Auth", "confidence": 0.90 }, { "pattern": "RBAC", "confidence": 0.75 } ], "performancePatterns": [ { "pattern": "Client-side Caching", "confidence": 0.80 }, { "pattern": "Lazy Loading", "confidence": 0.65 } ] }, "evolutionTimeline": [ { "version": "v1.0.0", "date": "2025-03-15", "features": ["Basic auth", "Product listing", "Simple cart"], "reason": "MVP validation – get users first" }, { "version": "v2.0.0", "date": "2025-06-20", "features": ["Stripe payments", "User profiles", "Order history"], "reason": "Monetization – users asked for checkout" }, { "version": "v3.0.0", "date": "2025-11-10", "features": ["AI recommendations", "Analytics dashboard", "Multi-vendor support"], "reason": "Competitive edge – differentiate from Shopify" } ], "thinkingProcess": { "whyThisStack": "Next.js chosen for SSR + API routes in one codebase. Prisma for type-safe DB. Stripe because it's easiest payment integration.", "tradeOffs": [ { "decision": "Monolith over microservices", "reason": "Solo developer, faster to build and deploy", "regret": false }, { "decision": "MongoDB initially, then migrated to PostgreSQL", "reason": "Mongo was flexible for rapid iteration, but relational data needed for orders/payments", "regret": true } ], "priorities": ["Speed to market", "User feedback", "MVP validation"], "constraints": ["Solo developer", "$0 initial budget", "3-month deadline"], "regrets": ["Should have used TypeScript from day 1", "Should have added tests before v2"], "whatWouldChange": "Use tRPC for type-safe APIs, add testing framework early, use Vercel from start" }, "scores": { "innovation": 78, "technicalDebt": 45, "maintainability": 62, "scalability": 55 }, "futureVersions": [ { "version": "AI-Enhanced Commerce", "enhancement": "Add GPT-4 powered product descriptions, AI chatbot for support, predictive inventory", "effort": "MEDIUM", "impact": "HIGH" }, { "version": "Headless Commerce API", "enhancement": "Extract core as API-only, support mobile apps, IoT, voice commerce", "effort": "HIGH", "impact": "HIGH" } ] }
```

6. Database Schema — Archaeology-Specific

```
model ArchaeologyReport { id String @id @default(uuid()) projectId String @unique
project Project @relation(fields: [projectId], references: [id]) // Input metadata
inputType String // CODE_SNIPPET | GITHUB_URL | WEBSITE_URL inputUrl String? // For
GitHub/Website inputCode String? // For raw code (stored as text) // Agent outputs
codeAnalysis Json // CodeParserAgent output patterns Json // PatternDetectorAgent
output evolutionTimeline Json // IdeaHistorianAgent output thinkingProcess Json //
DevPsychologistAgent output scores Json // ScorerAgent output futureVersions Json
// EvolverAgent output // Raw analysis for debugging rawAnalysis String? // Full
LLM response stored // Vector embedding of the code for similarity search embedding
Unsupported("vector(1536)") createdAt DateTime @default(now()) updatedAt DateTime
@updatedAt }
```

7. LangChain SequentialChain — 6 Agents

```
// Code Archaeology – 6-Agent SequentialChain import { SequentialChain, LLMChain }
from "langchain/chains"; import { OpenAI } from "langchain/llms/openai"; // Agent
1: Code Parser const codeParserChain = new LLMChain({ llm: new OpenAI({
temperature: 0.3 }), prompt: codeParserPrompt, outputKey: "code_analysis" }); //
Agent 2: Pattern Detector const patternDetectorChain = new LLMChain({ llm: new
OpenAI({ temperature: 0.3 }), prompt: patternDetectorPrompt, outputKey: "patterns"
}); // Agent 3: Idea Historian const ideaHistorianChain = new LLMChain({ llm: new
OpenAI({ temperature: 0.5 }), prompt: ideaHistorianPrompt, outputKey:
"evolution_timeline" }); // Agent 4: Dev Psychologist const devPsychologistChain =
new LLMChain({ llm: new OpenAI({ temperature: 0.4 }), prompt:
devPsychologistPrompt, outputKey: "thinking_process" }); // Agent 5: Scorer const
scorerChain = new LLMChain({ llm: new OpenAI({ temperature: 0.2 }), prompt:
scorerPrompt, outputKey: "scores" }); // Agent 6: Evolver const evolverChain = new
LLMChain({ llm: new OpenAI({ temperature: 0.6 }), prompt: evolverPrompt, outputKey:
"future_versions" }); // Complete Pipeline const archaeologyPipeline = new
SequentialChain({ chains: [ codeParserChain, patternDetectorChain,
ideaHistorianChain, devPsychologistChain, scorerChain, evolverChain ],
inputVariables: ["code_input", "input_type", "user_profile"], outputVariables: [
"code_analysis", "patterns", "evolution_timeline", "thinking_process", "scores",
"future_versions" ] }); // Execute const result = await archaeologyPipeline.call({
code_input: githubRepoData, // or rawCode or websiteData input_type: "GITHUB_URL",
user_profile: JSON.stringify(userProfile) });
```


8. API Endpoints

Method	Endpoint	Description
POST	/api/archaeology/analyze	Full 6-agent pipeline
POST	/api/archaeology/parse-code	CodeParserAgent only
POST	/api/archaeology/detect-patterns	PatternDetector only
POST	/api/archaeology/history	IdeaHistorian only
POST	/api/archaeology/psychology	DevPsychologist only
POST	/api/archaeology/score	Scorer only
POST	/api/archaeology/evolve	Evolver only
GET	/api/archaeology/report/:id	Get complete report

9. Interview Talking Points

Why 6 Agents Instead of 1?

Ek agent se sab kuch karne se output generic hota hai. 6 agents se har level pe depth milti hai: CodeParser = technical, PatternDetector = architectural, IdeaHistorian = temporal, DevPsychologist = cognitive. Ye modular hai — koi bhi agent replace/upgrade kar sakte hain.

What Makes This 'Reverse Engineering'?

Normal code analysis batata hai 'ye code kya karta hai'. Hum batate hain 'developer ne ye kyun socha'. Example: Code mein MongoDB se PostgreSQL migration dikhi → historian ne v1→v2→v3 timeline banaya → psychologist ne 'flexible schema → relational data need' trade-off infer kiya.

How Do You Handle GitHub Private Repos?

User se GitHub OAuth token mangte hain. Token ke saath private repo access possible. Agar token nahi, toh sirf public repos analyze kar sakte hain. Security: token server-side store nahi karte, session mein temporarily rakhte hain.

What's the 'Wow' Factor for Judges?

User koi random GitHub repo daale, system batata hai: 'Ye developer ne pehle MongoDB use ki, phir migrate kiya PostgreSQL pe kyunki orders relational data tha. Usne TypeScript nahi use ki — regret hai. Agar ye aaj banata, toh tRPC + Vercel use karta.' Ye 'mind reading' hai.

— End of Guide 3: Code Archaeology PRD —